Classification d'intentions sur le corpus SLURP

TP Dialogue oral et IA, UPSSITECH 5A SRI
Léo Ahmed-Mushtaq (SBERT) · Axel Sichi (BERT)

1. Contexte et démarche

L'objectif est de classer 7 intentions du corpus SLURP (alarm_set, alarm_remove, alarm_query, calendar_set, calendar_remove, calendar_query, news_query) à partir d'embeddings de la transcription, avec Orange Data Mining. Le jeu d'apprentissage compte 2 559 énoncés et le jeu de test 617.

Deux représentations ont été étudiées : BERT (Axel) et Sentence-BERT (Léo). Chaîne de traitement commune : CSV File Import → Select Columns (cible = intent) → ACP → classifieurs → Test and Score → Confusion Matrix. Faute de temps, WavLM et l'apprentissage non supervisé n'ont pas été traités.

2. Prétraitements

2.1 Distribution des données

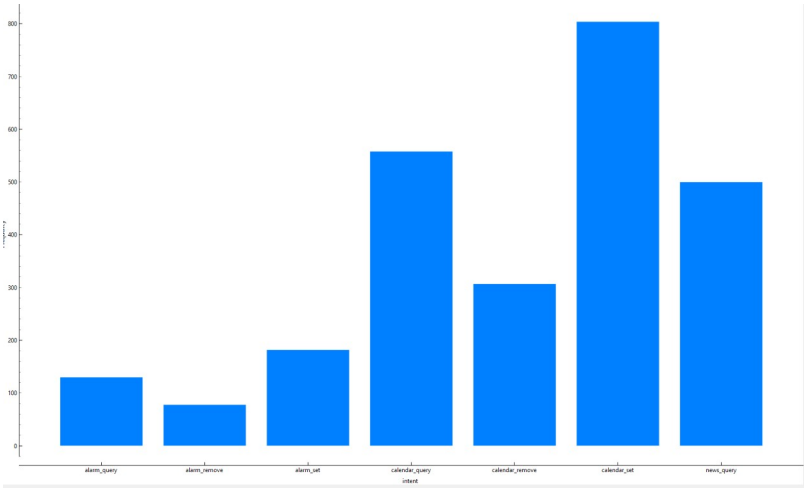


Figure 1 : Distribution des intentions dans le fichier d'apprentissage

Les classes sont déséquilibrées : calendar_set domine avec 804 énoncés (31,42 %), alors qu'alarm_remove n'en compte que 78 (3,05 %). Le jeu de test suit la même tendance (208 calendar_set sur 617). Les classes rares sont donc moins bien apprises, et la précision globale (CA) peut masquer de mauvais résultats sur ces classes : nous nous appuyons aussi sur le F1, le MCC et la matrice de confusion.

2.2 Réduction de dimension (ACP)

Une ACP a été appliquée avant les classifieurs afin de réduire la dimension des vecteurs. Chaque classifieur a été évalué avec et sans ACP : les écarts de CA sont au plus de 0,01 (tableaux 1 et 4). L'ACP réduit donc le coût de calcul sans dégrader nettement les résultats.

3. Métriques utilisées

- CA (accuracy) : part d'énoncés correctement classés ; trompeuse en cas de classes déséquilibrées.
- Précision, rappel, F1 : qualité des prédictions et couverture de chaque classe (moyenne pondérée) ; le F1 est leur moyenne harmonique.
- AUC : capacité à séparer chaque classe des autres, indépendamment d'un seuil.
- MCC : corrélation entre prédictions et réalité (1 = parfait, 0 = hasard), robuste au déséquilibre.

- Matrice de confusion : localise les erreurs entre intentions. Les pourcentages affichés sont normalisés par classe prédite (colonnes).

4. Résultats avec SBERT

4.1 Classifieurs avec paramètres par défaut

Tableau 1 : CA et MCC sur le jeu de test, avec et sans ACP

Modèle	CA sans ACP	CA avec ACP	MCC sans ACP	MCC avec ACP
Logistic Regression	0,930	0,930	0,911	0,911
SVM	0,927	0,927	0,907	0,907
Stochastic Gradient Descent	0,887	0,887	0,857	0,858
kNN	0,882	0,882	0,850	0,850
Naive Bayes	0,859	0,859	0,820	0,820
Random Forest	0,838	0,828	0,793	0,781

La régression logistique (0,930) et le SVM (0,927) sont nettement devant les autres méthodes. Par souci de temps, nous n'avons conservé que ces deux modèles pour la suite.

4.2 Réglage des hyperparamètres

Tableau 2 : Hyperparamètres (SBERT)

Modèle	Par défaut	Retenus
Logistic Regression	Régularisation L2, C = 1, pas d'équilibrage des classes	Régularisation L2, C = 5, équilibrage des classes activé
SVM	C = 1, ε = 0,1, noyau RBF, tolérance 0,001, 100 itérations max	C = 3,5, ε = 0,1, noyau RBF, tolérance 0,001, 100 itérations max

Tableau 3 : Résultats avant/après réglage (SBERT, avec ACP)

Modèle	CA	F1	MCC	AUC
Logistic Regression (défaut)	0,930	0,930	0,911	0,991
Logistic Regression (réglée)	0,934	0,934	0,916	0,992
SVM (défaut)	0,927	0,927	0,907	0,991
SVM (réglé)	0,942	0,942	0,926	0,992

Le réglage améliore le SVM de 1,5 point de CA (0,927 → 0,942) et la régression logistique de 0,4 point : le SVM devient le meilleur modèle SBERT.

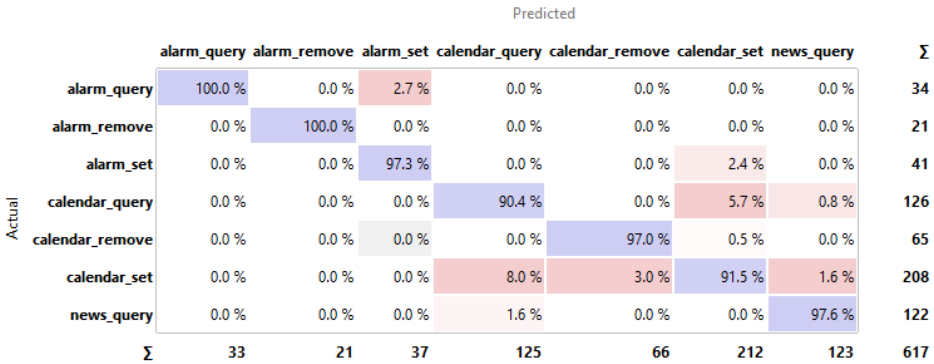


Figure 2 : Matrice de confusion du SVM réglé (SBERT)

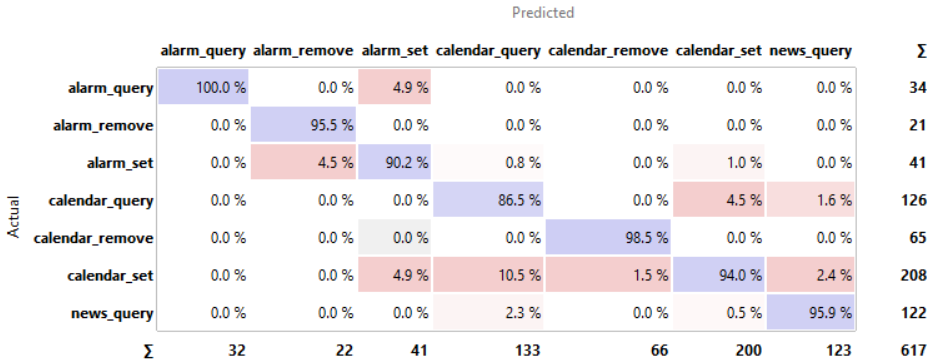


Figure 3 : Matrice de confusion de la régression logistique réglée (SBERT)

Les intentions alarm_query et alarm_remove sont reconnues à 100 % par le SVM. Les erreurs se concentrent sur les intentions calendar : parmi les énoncés prédits calendar_query, 8 % sont en réalité calendar_set, et calendar_set est aussi confondu avec calendar_remove (3 %). Ces intentions partagent probablement le même vocabulaire (appointment, reminder, date) et ne diffèrent que par l'action demandée, que l'embedding de phrase distingue mal ; calendar_set, classe la plus fréquente, concentre ces confusions. La régression logistique ajoute de petites confusions avec alarm_set (environ 5 %).

5. Résultats avec BERT

Tableau 4 : CA et MCC sur le jeu de test, avec et sans ACP (paramètres par défaut)

Modèle	CA sans ACP	CA avec ACP	MCC sans ACP	MCC avec ACP
SVM	0,948	0,948	0,934	0,934
Neural Network	0,932	0,932	0,913	0,913
Stochastic Gradient Descent	0,904	0,912	0,878	0,889
Logistic Regression	0,896	0,896	0,868	0,868
kNN	0,893	0,893	0,864	0,864
Naive Bayes	0,831	0,831	0,789	0,789
Random Forest	0,793	0,794	0,733	0,736

Le SVM (0,948) et le réseau de neurones (0,932) se détachent ; Naive Bayes et Random Forest sont les moins performants. Le SVM et la régression logistique ont ensuite été réglés.

Tableau 5 : Hyperparamètres et résultats après réglage (BERT)

Modèle	Hyperparamètres retenus	CA	F1	MCC	AUC
SVM	C = 1,6 ; noyau RBF ; ε = 0,1 ; tolérance 0,0005 ; 100 itérations max	0,953	—	0,940	0,995
Logistic Regression	L2, C = 30, pas d'équilibrage des classes	0,940	—	0,924	0,999

Le réglage gagne 0,5 point pour le SVM (0,948 → 0,953) et 4,4 points pour la régression logistique (0,896 → 0,940) : avec des vecteurs BERT de grande dimension, une régularisation plus faible (C = 30) est nettement préférable à C = 1. Les F1, précision et rappel de ces captures ne sont pas calculés avec la même moyenne que dans le reste du rapport : ils ne sont pas comparables et ne sont pas repris ici. Un affinage avec le jeu de dev donne des scores plus bas (SVM 0,860 ; régression logistique 0,807).

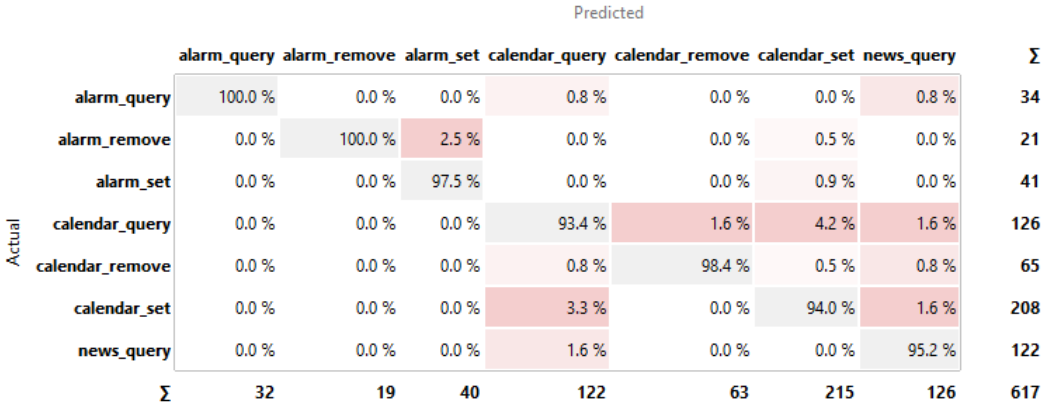


Figure 4 : Matrice de confusion du SVM réglé (BERT)

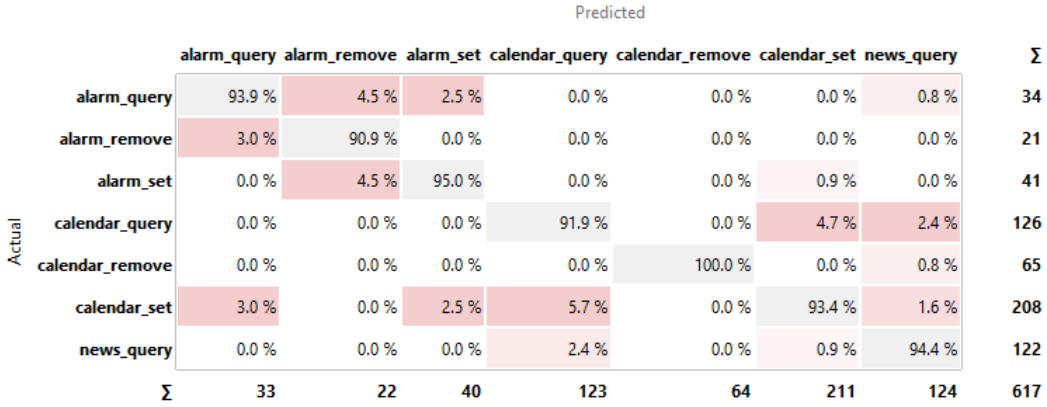


Figure 5 : Matrice de confusion de la régression logistique réglée (BERT)

Le schéma d'erreurs est le même qu'avec SBERT : les confusions portent surtout sur calendar_query et calendar_set (environ 3 à 6 %). La régression logistique réglée fait en plus quelques confusions entre intentions alarm (de l'ordre de 3 à 4,5 %).

6. Comparaison et conclusion

Tableau 6 : Meilleur modèle par représentation

Représentation	Meilleur modèle et hyperparamètres	CA	MCC	AUC
BERT	SVM RBF, C = 1,6, tolérance 0,0005	0,953	0,940	0,995
SBERT	SVM RBF, C = 3,5, tolérance 0,001	0,942	0,926	0,992

Dans les deux cas, la meilleure méthode est le SVM à noyau RBF, avec la régression logistique juste derrière. BERT donne les meilleurs résultats (+1,1 point de CA), mais l'écart représente environ 7 énoncés sur 617 : il est peu significatif. SBERT fournit directement un vecteur par phrase de plus petite dimension, donc moins coûteux à traiter.

Limites : WavLM et l'apprentissage non supervisé n'ont pas été étudiés ; seuls la régression logistique et le SVM ont été réglés pour SBERT (le réseau de neurones n'y a pas été testé) ; le déséquilibre des classes n'a été corrigé que pour la régression logistique SBERT. Les erreurs restantes concernent surtout la distinction entre les intentions calendar_set, calendar_query et calendar_remove.

Annexe : Utilisation de l'IA générative

Pour réaliser ce rapport, nous sommes partis directement de nos expériences sur Orange Data Mining (ACP, modélisation, réglage des paramètres...). Nous avons sollicité Claude (Anthropic), principalement pour retravailler la forme et la présentation de nos données pour ce rapport.